Deep Learning: Labwork 1

Gradient Descent From Scratch

Prepared by:
Bui Dang Quang - Student ID: 2540092

May 9, 2026

1 Introduction

This lab work involves a practical session for building the Gradient Descent from scratch. The project will explain how to implement this algorithm in Python code based on only raw mathematical operations, in which Python will not use any open library. We test our custom algorithm on the scenarios that interact with the function f(x) = x^2

2 Implementaion

In this scenario, we define a square function (quadratic function). The objective is to analyze how the variable x changes when subjected to the gradient descent optimization process.
Having the function and its partial derivative, which is used as the gradient to update x in our simulation, are mathematically defined as:

$$f(x) = x^2 \tag{1}$$

$$\frac{\partial f}{\partial x} = 2x \tag{2}$$

The gradient is then being trained simultaneously as it is used to update the variable x (this variable is initialized to 1, which is a starting point). This variable is mentioned as a way to demonstrate how the gradient descent algorithm iteratively navigates toward the local minimum.

At each iteration, the gradient is evaluated at the current value of x. Because the gradient points in the direction of the steepest ascent, the algorithm subtracts a fraction of this gradient from x to move downwards toward the minimum. The update rule applied at each epoch is:

$$x = x - lr * \frac{\partial f}{\partial x} \tag{3}$$

The algorithm loops continuously until a stopping criterion is met. In this simulation, convergence is achieved when the absolute value of the gradient falls below a predefined threshold of 0.5. Once $|\frac{\partial f}{\partial x}| < 0.5$, the algorithm assumes it has successfully approximated the local minimum and terminates the loop.

3 Result

As detailed in Table 1, the algorithm with an initial starting point of $x_0 = 1.0$, a learning rate of $lr = 0.1$, successfully converged at Epoch 8 once the absolute value of the derivative fell below the threshold of 0.5. The final approximated local minimum is located at $x \approx 0.1678$, yielding a corresponding function value of $f(x) \approx 0.0281$.

3.1 Impact of the Learning Rate (η)

To further analyze the behavior of the gradient descent optimization process, we experimented with three additional learning rates: $\eta = 0.01$, $\eta = 0.001$, and $\eta = 1.0$. The

Table 1: Gradient Descent Optimization Path for $f(x) = x^2$ ($lr = 0.1$)

Epoch	$\boldsymbol{x}$	$\boldsymbol{f(x)}$	$\frac{df}{dx}$
1	0.8000	0.6400	2.0000
2	0.6400	0.4096	1.6000
3	0.5120	0.2621	1.2800
4	0.4096	0.1678	1.0240
5	0.3277	0.1074	0.8192
6	0.2621	0.0687	0.6554
7	0.2097	0.0440	0.5243
8	0.1678	0.0281	0.4194

initial starting point ($x_0 = 1$) and the convergence threshold (0.5) remained constant. The summarized results of these experiments are presented in Table 2.

Table 2: Comparison of Gradient Descent Performance Across Different Learning Rates

Learning Rate (η)	**Epochs to Converge**	**Final $\boldsymbol{x}$**	**Final $\boldsymbol{f(x)}$**	**Observed Behavior**
0.1	8	0.1678	0.0281	Fast, stable convergence
0.01	70	0.2431	0.0591	Slow, steady convergence
0.001	694	0.2492	0.0621	Extremely slow convergence
1.0	N/A (Does not converge)	N/A	N/A	Endless oscillation

Case 1: Small Learning Rates ($lr = 0.01$ and $lr = 0.001$)
When the learning rate is reduced to 0.01, the algorithm requires significantly more iterations (70 epochs) to reach the stopping criterion. The step sizes become much smaller, making the descent cautious but computationally expensive. This effect is drastically magnified when $\eta = 0.001$, requiring 694 epochs to converge.

Interestingly, the final approximated x values for these smaller learning rates (0.2431 and 0.2492) are slightly further from the true minimum of 0 compared to the $lr = 0.1$ run (0.1678). This occurs because with very small step sizes, the algorithm strictly stops the moment the gradient dips below the 0.5 threshold (at $x \approx 0.25$). A larger learning rate can move towards slightly in its final step, landing closer to the true minimum before the loop breaks.

Case 2: Oversized Learning Rate ($lr = 1.0$)
When the learning rate is set too high ($lr = 1.0$), the algorithm entirely fails to converge. The system falls into an endless run between $x = 1.0$ and $x = -1.0$. We can mathematically explain this behavior using the update rule:

$$x_{new} = x_{old} - lr * 2x_{old}$$

Substituting $lr = 1.0$:

$$x_{new} = x_{old} - 2x_{old} = -x_{old}$$

Consequently, an initial state of $x = 1$ updates to $x = -1$, which then updates back to $x = 1$ in the subsequent epoch. The gradient constantly flips between 2.0 and -2.0, never decreasing in magnitude.